Assignment 9 - Assignment

Topic: Decision Tree

Dataset: Assignment_9b.csv

age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
60	1	3	145	233	1	0	150	0	2.3	0	0	1	1
35	1	2	130	250	0	1	187	0	3.5	0	0	2	1
41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
55	1	1	120	236	0	1	178	0	0.8	2	0	2	1
56	0	0	120	354	0	1	163	1	0.6	2	0	2	1
55	1	0	140	192	0	1	148	0	0.4	1	0	1	1
56	0	1	140	294	0	0	153	0	1.3	1	0	2	1
44	1	1	120	263	0	1	173	0	0	2	0	3	1
52	1	2	172	199	1	1	162	0	0.5	2	0	3	1
57	1	2	150	168	0	1	174	0	1.6	2	0	2	1
54	1	0	140	239	0	1	160	0	1.2	2	0	2	1
48	0	2	130	275	0	1	139	0	0.2	2	0	2	1
49	1	1	130	266	0	1	171	0	0.6	2	0	2	1
64	1	3	110	211	0	0	144	1	1.8	1	0	2	1
55	0	3	150	283	1	0	162	0	1	2	0	2	1
50	0	2	120	219	0	1	158	0	1.6	1	0	2	1
58	0	2	120	340	0	1	172	0	0	2	0	2	1
66	0	3	150	226	0	1	114	0	2.6	0	0	2	1
40	1	0	150	247	0	1	171	0	1.5	2	0	2	1
69	0	3	140	239	0	1	151	0	1.8	2	2	2	1
59	1	0	135	234	0	1	161	0	0.5	1	0	3	1
44	1	2	130	233	0	1	179	1	0.4	2	0	2	1
42	1	0	140	226	0	1	178	0	0	2	0	2	1
61	1	2	150	243	1	1	137	1	1	1	0	2	1
40	1	3	140	199	0	1	178	1	1.4	2	0	3	1
71	0	1	160	302	0	1	162	0	0.4	2	2	2	1
59	1	2	150	212	1	1	157	0	1.6	2	0	2	1
51	1	2	110	175	0	1	123	0	0.6	2	0	2	1
65	0	2	140	417	1	0	157	0	0.8	2	1	2	1
53	1	2	130	197	1	0	152	0	1.2	0	0	2	1
41	0	1	105	198	0	1	168	0	0	2	1	2	1
65	1	0	120	177	0	1	140	0	0.4	2	0	3	1
44	1	1	130	219	0	0	188	0	0	2	0	2	1
54	1	2	125	273	0	0	152	0	0.5	0	1	2	1
51	1	3	125	213	0	0	125	1	1.4	2	1	2	1
46	0	2	142	177	0	0	160	1	1.4	0	0	2	1
54	0	2	135	304	1	1	170	0	0	2	0	2	1
54	1	2	150	232	0	0	165	0	1.6	2	0	3	1
65	0	2	155	269	0	1	148	0	0.8	2	0	2	1
65	0	2	160	360	0	0	151	0	0.8	2	0	2	1
51	0	2	140	308	0	0	142	0	1.5	2	1	2	1
48	1	1	130	245	0	0	180	0	0.2	1	0	2	1
45	1	0	104	208	0	0	148	1	3	1	0	2	1
53	0	0	130	264	0	0	143	0	0.4	1	0	2	1
39	1	2	140	321	0	0	182	0	0	2	0	2	1
52	1	1	120	325	0	1	172	0	0.2	2	0	2	1
44	1	2	140	235	0	0	180	0	0	2	0	2	1
47	1	2	138	257	0	0	156	0	0	2	0	2	1
53	0	2	128	216	0	0	115	0	0	2	0	0	1
53	0	0	138	234	0	0	160	0	0	2	0	2	1
51	0	2	130	256	0	0	149	0	0.5	2	0	2	1
66	1	0	120	302	0	0	151	0	0.4	1	0	2	1
62	1	2	130	231	0	1	146	0	1.8	1	3	3	1
44	0	2	108	141	0	1	175	0	0.6	1	0	2	1
63	0	2	135	252	0	0	172	0	0	2	0	2	1
52	1	1	134	201	0	1	158	0	0.8	2	1	2	1
48	1	0	122	222	0	0	186	0	0	2	0	2	1
45	1	0	115	260	0	0	185	0	0	2	0	2	1
34	1	3	118	182	0	0	174	0	0	2	0	2	1
57	0	0	128	303	0	0	159	0	0	2	1	2	1
71	0	2	110	265	1	0	130	0	0	2	1	2	1

age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
54	1	1	108	309	0	1	156	0	0	2	0	3	1
52	1	3	118	186	0	0	190	0	0	1	0	1	1
41	1	1	135	203	0	1	132	0	0	1	0	1	1
58	1	2	140	211	1	0	165	0	0	2	0	2	1
35	0	0	138	183	0	1	182	0	1.4	2	0	2	1
51	1	2	100	222	0	1	143	1	1.2	1	0	2	1
45	0	1	130	234	0	0	175	0	0.6	1	0	2	1
44	1	1	120	220	0	1	170	0	0	2	0	2	1
62	0	0	124	209	0	1	163	0	0	2	0	2	1
54	1	2	120	258	0	0	147	0	0.4	1	0	3	1
51	1	2	94	227	0	1	154	1	0	2	1	3	1
29	1	1	130	204	0	0	202	0	0	2	0	2	1
51	1	0	140	261	0	0	186	1	0	2	0	2	1
43	0	2	122	213	0	1	165	0	0.2	1	0	2	1
55	0	1	135	250	0	0	161	0	1.4	1	0	2	1
51	1	2	125	245	1	0	166	0	2.4	1	0	2	1
59	1	1	140	221	0	1	164	1	0	2	0	2	1
52	1	1	128	205	1	1	184	0	0	2	0	2	1
58	1	2	105	240	0	0	154	1	0.6	1	0	3	1
41	1	2	112	250	0	1	179	0	0	2	0	2	1
45	1	1	128	308	0	0	170	0	0	2	0	2	1
60	0	2	102	318	0	1	160	0	0	2	1	2	1
52	1	3	152	298	1	1	178	0	1.2	1	0	3	1
42	0	0	102	265	0	0	122	0	0.6	1	0	2	1
67	0	2	115	564	0	0	160	0	1.6	1	0	3	1
68	1	2	118	277	0	1	151	0	1	2	1	3	1
46	1	1	101	197	1	1	156	0	0	2	0	3	1
54	0	2	110	214	0	1	158	0	1.6	1	0	2	1
58	0	0	100	248	0	0	122	0	1	1	0	2	1
48	1	2	124	255	1	1	175	0	0	2	2	2	1
57	1	0	132	207	0	1	168	1	0	2	0	3	1
52	1	2	138	223	0	1	169	0	0	2	4	2	1
54	0	1	132	288	1	0	159	1	0	2	1	2	1
45	0	1	112	160	0	1	138	0	0	1	0	2	1
53	1	0	142	226	0	0	111	1	0	2	0	3	1
62	0	0	140	394	0	0	157	0	1.2	1	0	2	1
52	1	0	108	233	1	1	147	0	0.1	2	3	3	1
43	1	2	130	315	0	1	162	0	1.9	2	1	2	1
53	1	2	130	246	1	0	173	0	0	2	3	2	1
42	1	3	148	244	0	0	178	0	0.8	2	2	2	1
59	1	3	178	270	0	0	145	0	4.2	0	0	3	1
63	0	1	140	195	0	1	179	0	0	2	2	2	1
42	1	2	120	240	1	1	194	0	0.8	0	0	3	1
50	1	2	129	196	0	1	163	0	0	2	0	2	1
68	0	2	120	211	0	0	115	0	1.5	1	0	2	1
69	1	3	160	234	1	0	131	0	0.1	1	1	2	1
45	0	0	138	236	0	0	152	1	0.2	1	0	2	1
50	0	1	120	244	0	1	162	0	1.1	2	0	2	1
50	0	0	110	254	0	0	159	0	0	2	0	2	1
64	0	0	180	325	0	1	154	1	0	2	0	2	1
57	1	2	150	126	1	1	173	0	0.2	2	1	3	1
64	0	2	140	313	0	1	133	0	0.2	2	0	3	1
43	1	0	110	211	0	1	161	0	0	2	0	3	1
55	1	1	130	262	0	1	155	0	0	2	0	2	1
37	0	2	120	215	0	1	170	0	0	2	0	2	1
41	1	2	130	214	0	0	168	0	2	1	0	2	1
56	1	3	120	193	0	0	162	0	1.9	1	0	3	1
46	0	1	105	204	0	1	172	0	0	2	0	2	1
46	0	0	138	243	0	0	152	1	0	1	0	2	1
64	0	0	130	303	0	1	122	0	2	1	2	2	1
59	1	0	138	271	0	0	182	0	0	2	0	2	1
41	0	2	112	268	0	0	172	1	0	2	0	2	1
54	0	2	108	267	0	0	167	0	0	2	0	2	1
39	0	2	94	199	0	1	179	0	0	2	0	2	1
34	0	1	118	210	0	1	192	0	0.7	2	0	2	1
47	1	0	112	204	0	1	143	0	0.1	2	0	2	1
67	0	2	152	277	0	1	172	0	0	2	1	2	1
52	0	2	136	196	0	0	169	0	0.1	1	0	2	1

age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
74	0	1	120	269	0	0	121	1	0.2	2	1	2	1
54	0	2	160	201	0	1	163	0	0	2	1	2	1
49	0	1	134	271	0	1	162	0	0	1	0	2	1
42	1	1	120	295	0	1	162	0	0	2	0	2	1
41	1	1	110	235	0	1	153	0	0	2	0	2	1
41	0	1	126	306	0	1	163	0	0	2	0	2	1
49	0	0	130	269	0	1	163	0	0	2	0	2	1
60	0	2	120	178	1	1	96	0	0	2	0	2	1
62	1	1	128	208	1	0	140	0	0	2	0	2	1
57	1	0	110	201	0	1	126	1	1.5	1	0	1	1
64	1	0	128	263	0	1	105	1	0.2	1	1	3	1
51	0	2	120	295	0	0	157	0	0.6	2	0	2	1
43	1	0	115	303	0	1	181	0	1.2	1	0	2	1
42	0	2	120	209	0	1	173	0	0	1	0	2	1
67	0	0	106	223	0	1	142	0	0.3	2	2	2	1
76	0	2	140	197	0	2	116	0	1.1	1	0	2	1
70	1	1	156	245	0	0	143	0	0	2	0	2	1
44	0	2	118	242	0	1	149	0	0.3	1	1	2	1
60	0	3	150	240	0	1	171	0	0.9	2	0	2	1
44	1	2	120	226	0	1	169	0	0	2	0	2	1
42	1	2	130	180	0	1	150	0	0	2	0	2	1
66	1	0	160	228	0	0	138	0	2.3	2	0	1	1
71	0	0	112	149	0	1	125	0	1.6	1	0	2	1
64	1	3	170	227	0	0	155	0	0.6	1	0	3	1
66	0	2	146	278	0	0	152	0	0	1	1	2	1
39	0	2	138	220	0	1	152	0	0	1	0	2	1
58	0	0	130	197	0	1	131	0	0.6	1	0	2	1
47	1	2	130	253	0	1	179	0	0	2	0	2	1
35	1	1	122	192	0	1	174	0	0	2	0	2	1
58	1	1	125	220	0	1	144	0	0.4	1	4	3	1
56	1	1	130	221	0	0	163	0	0	2	0	3	1
56	1	1	120	240	0	1	169	0	0	0	0	2	1
55	0	1	132	342	0	1	166	0	1.2	2	0	2	1
41	1	1	120	157	0	1	182	0	0	2	0	2	1
38	1	2	138	175	0	1	173	0	0	2	4	2	1
38	1	2	138	175	0	1	173	0	0	2	4	2	1
67	1	0	160	286	0	0	108	1	1.5	1	3	2	0
67	1	0	120	229	0	0	129	1	2.6	1	2	3	0
62	0	0	140	268	0	0	160	0	3.6	0	2	2	0
63	1	0	130	254	0	0	147	0	1.4	1	1	3	0
53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
56	1	2	130	256	1	0	142	1	0.6	1	1	1	0
48	1	1	110	229	0	1	168	0	1	0	0	3	0
58	1	1	120	284	0	0	160	0	1.8	1	0	2	0
58	1	2	132	224	0	0	173	0	3.2	2	2	3	0
60	1	0	130	206	0	0	132	1	2.4	1	2	3	0
40	1	0	110	167	0	0	114	1	2	1	0	3	0
60	1	0	117	230	1	1	160	1	1.4	2	2	3	0
64	1	2	140	335	0	1	158	0	0	2	0	2	0
43	1	0	120	177	0	0	120	1	2.5	1	0	3	0
57	1	0	150	276	0	0	112	1	0.6	1	1	1	0
55	1	0	132	353	0	1	132	1	1.2	1	1	3	0
65	0	0	150	225	0	0	114	0	1	1	3	3	0
61	0	0	130	330	0	0	169	0	0	2	0	2	0
58	1	2	112	230	0	0	165	0	2.5	1	1	3	0
50	1	0	150	243	0	0	128	0	2.6	1	0	3	0
44	1	0	112	290	0	0	153	0	0	2	1	2	0
60	1	0	130	253	0	1	144	1	1.4	2	1	3	0
54	1	0	124	266	0	0	109	1	2.2	1	1	3	0
50	1	2	140	233	0	1	163	0	0.6	1	1	3	0
41	1	0	110	172	0	0	158	0	0	2	0	3	0
51	0	0	130	305	0	1	142	1	1.2	1	0	3	0
58	1	0	128	216	0	0	131	1	2.2	1	3	3	0
54	1	0	120	188	0	1	113	0	1.4	1	1	3	0
60	1	0	145	282	0	0	142	1	2.8	1	2	3	0
60	1	2	140	185	0	0	155	0	3	1	0	2	0
59	1	0	170	326	0	0	140	1	3.4	0	0	3	0
46	1	2	150	231	0	1	147	0	3.6	1	0	2	0

age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
67	1	0	125	254	1	1	163	0	0.2	1	2	3	0
62	1	0	120	267	0	1	99	1	1.8	1	2	3	0
65	1	0	110	248	0	0	158	0	0.6	2	2	1	0
44	1	0	110	197	0	0	177	0	0	2	1	2	0
60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
58	1	0	150	270	0	0	111	1	0.8	2	0	3	0
68	1	2	180	274	1	0	150	1	1.6	1	0	3	0
62	0	0	160	164	0	0	145	0	6.2	0	3	3	0
52	1	0	128	255	0	1	161	1	0	2	1	3	0
59	1	0	110	239	0	0	142	1	1.2	1	1	3	0
60	0	0	150	258	0	0	157	0	2.6	1	2	3	0
49	1	2	120	188	0	1	139	0	2	1	3	3	0
59	1	0	140	177	0	1	162	1	0	2	1	3	0
57	1	2	128	229	0	0	150	0	0.4	1	1	3	0
61	1	0	120	260	0	1	140	1	3.6	1	1	3	0
39	1	0	118	219	0	1	140	0	1.2	1	0	3	0
61	0	0	145	307	0	0	146	1	1	1	0	3	0
56	1	0	125	249	1	0	144	1	1.2	1	1	2	0
43	0	0	132	341	1	0	136	1	3	1	0	3	0
54	1	2	130	263	0	1	97	0	1.2	1	1	3	0
63	1	0	130	330	1	0	132	1	1.8	2	3	3	0
60	1	0	135	254	0	0	127	0	2.8	1	1	3	0
48	1	0	130	256	1	0	150	1	0	2	2	3	0
63	1	0	150	407	0	0	154	0	4	1	3	3	0
55	1	0	140	217	0	1	111	1	5.6	0	0	3	0
64	1	3	138	282	1	0	174	0	1.4	1	1	2	0
56	0	0	200	288	1	0	133	1	4	0	2	3	0
50	1	0	110	239	0	1	126	1	2.8	1	1	3	0
70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
62	1	1	120	281	0	0	103	0	1.4	1	1	3	0
35	1	0	120	198	0	1	130	1	1.6	1	0	3	0
59	1	3	170	288	0	0	159	0	0.2	1	0	3	0
64	1	2	125	309	0	1	131	1	1.8	1	0	3	0
47	1	2	108	243	0	1	152	0	0	2	0	2	0
57	1	0	165	289	1	0	124	0	1	1	3	3	0
55	1	0	160	289	0	0	145	1	0.8	1	1	3	0
64	1	0	120	246	0	0	96	1	2.2	0	1	2	0
70	1	0	130	322	0	0	109	0	2.4	1	3	2	0
51	1	0	140	299	0	1	173	1	1.6	2	0	3	0
58	0	0	125	300	0	0	171	0	0	2	2	3	0
60	1	0	140	293	0	0	170	0	1.2	1	2	3	0
77	0	0	125	304	0	0	162	1	0	2	3	2	0
35	1	0	126	282	0	0	156	1	0	2	0	3	0
70	0	2	160	269	0	1	112	1	2.9	1	1	3	0
59	0	0	174	249	0	1	143	1	0	1	0	2	0
64	1	1	145	212	0	0	132	0	2	1	2	1	0
57	1	0	152	274	0	1	88	1	1.2	1	1	3	0
56	1	1	132	184	0	0	105	1	2.1	1	1	1	0
48	1	0	124	274	0	0	166	0	0.5	1	0	3	0
56	0	0	134	409	0	0	150	1	1.9	1	2	3	0
66	1	1	160	246	0	1	120	1	0	1	3	1	0
54	1	1	192	283	0	0	195	0	0	2	1	3	0
69	1	2	140	254	0	0	146	0	2	1	3	3	0
51	1	0	140	298	0	1	122	1	4.2	1	3	3	0
43	0	3	132	247	1	0	143	1	0.1	1	4	3	0
62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
67	1	0	100	299	0	0	125	1	0.9	1	2	2	0
59	1	3	160	273	0	0	125	0	0	2	0	2	0
45	1	0	142	309	0	0	147	1	0	1	3	3	0
58	1	1	128	259	0	0	130	1	3	1	2	3	0
50	1	1	144	200	0	0	126	1	0.9	1	0	3	0
62	0	1	150	244	0	1	154	1	1.4	1	0	2	0
38	1	3	120	231	0	1	182	1	3.8	1	0	3	0
66	0	0	178	228	1	1	165	1	1	1	2	3	0
52	1	0	112	230	0	1	160	0	0	2	1	2	0
53	1	0	123	282	0	1	95	1	2	1	2	3	0
63	0	0	108	269	0	1	169	1	1.8	1	2	2	0
54	1	0	110	206	0	0	108	1	0	1	1	2	0

age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
66	1	0	112	212	0	0	132	1	0.1	2	1	2	0
55	0	0	180	327	0	2	117	1	3.4	1	0	2	0
49	1	2	118	149	0	0	126	0	0.8	2	3	2	0
54	1	0	122	286	0	0	116	1	3.2	1	2	2	0
52	1	0	130	283	1	0	103	1	1.6	0	0	3	0
46	1	0	120	249	0	0	144	0	0.8	2	0	3	0
60	1	3	134	234	0	1	145	0	2.6	1	2	2	0
67	1	0	120	237	0	1	71	0	1	1	0	2	0
58	1	0	100	234	0	1	156	0	0.1	2	1	3	0
45	1	0	110	275	0	0	118	1	1	1	1	2	0
52	1	0	125	212	0	1	168	0	1	2	2	3	0
58	1	0	146	218	0	1	105	0	2	1	1	3	0
55	1	1	124	261	0	1	141	0	0.3	2	0	3	0
58	0	1	136	319	1	0	152	0	0	2	2	2	0
61	1	0	138	166	0	0	125	1	3.6	1	1	2	0
42	1	0	136	315	0	1	125	1	1.8	1	0	1	0
50	1	0	128	204	1	1	156	1	1	1	0	0	0
59	1	2	126	218	1	1	134	0	2.2	1	1	1	0
40	1	0	152	223	0	1	181	0	0	2	0	3	0
60	1	0	140	207	0	0	138	1	1.9	2	1	3	0
46	1	0	140	311	0	1	120	1	1.8	1	2	3	0
59	1	3	134	204	0	1	162	0	0.8	2	2	2	0
54	1	1	154	232	0	0	164	0	0	2	1	2	0
53	1	0	110	335	0	1	143	1	3	1	1	3	0

Question 1

Load the Heart Attack dataset from the CSV file. Display the first 10 rows and identify the input features and target variable.

Dataset: figshare.com

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report

# Load dataset
df = pd.read_csv('Assignment_9b.csv')

print('Dataset Shape:', df.shape)
print('\nFirst 10 Rows:')
print(df.head(10))
# Identify features and target
print('INPUT FEATURES (X):')
print('='*50)
feature_descriptions = {
    'age':      'Age of the patient (years)',
    'sex':      'Sex (1 = male, 0 = female)',
    'cp':       'Chest pain type (0-3)',
```

```python
    'trtbps':   'Resting blood pressure (mm Hg)',
    'chol':     'Serum cholesterol (mg/dl)',
    'fbs':      'Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)',
    'restecg':  'Resting ECG results (0-2)',
    'thalachh': 'Maximum heart rate achieved',
    'exng':     'Exercise-induced angina (1 = yes, 0 = no)',
    'oldpeak':  'ST depression induced by exercise relative to rest',
    'slp':      'Slope of peak exercise ST segment',
    'caa':      'Number of major vessels (0-4) colored by fluoroscopy',
    'thall':    'Thalassemia (0-3)'
}
for col, desc in feature_descriptions.items():
    print(f'  {col:10s}: {desc}')

print('\nTARGET VARIABLE (y):')
print('='*50)
print('  output: Heart attack likelihood (1 = more likely, 0 = less␣
 ↪likely)')
print('\nClass Distribution:')
print(df['output'].value_counts())
print(f'\nTotal samples: {len(df)}')
# Basic dataset info
print('Dataset Info:')
print(df.dtypes)
print('\nMissing Values:')
print(df.isnull().sum())
print('\nBasic Statistics:')
df.describe().round(2)
```

```
Dataset Shape: (289, 14)

First 10 Rows:
   age  sex  cp  trtbps  chol  fbs  restecg  thalachh  exng  oldpeak  slp  \
0   60    1   3     145   233    1        0       150     0      2.3    0
1   35    1   2     130   250    0        1       187     0      3.5    0
2   41    0   1     130   204    0        0       172     0      1.4    2
3   55    1   1     120   236    0        1       178     0      0.8    2
4   56    0   0     120   354    0        1       163     1      0.6    2
5   55    1   0     140   192    0        1       148     0      0.4    1
6   56    0   1     140   294    0        0       153     0      1.3    1
7   44    1   1     120   263    0        1       173     0      0.0    2
8   52    1   2     172   199    1        1       162     0      0.5    2
9   57    1   2     150   168    0        1       174     0      1.6    2

   caa  thall  output
0    0      1       1
1    0      2       1
2    0      2       1
3    0      2       1
4    0      2       1
5    0      1       1
```

```
6    0      2       1
7    0      3       1
8    0      3       1
9    0      2       1
INPUT FEATURES (X):
==================================================
  age       : Age of the patient (years)
  sex       : Sex (1 = male, 0 = female)
  cp        : Chest pain type (0-3)
  trtbps    : Resting blood pressure (mm Hg)
  chol      : Serum cholesterol (mg/dl)
  fbs       : Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)
  restecg   : Resting ECG results (0-2)
  thalachh  : Maximum heart rate achieved
  exng      : Exercise-induced angina (1 = yes, 0 = no)
  oldpeak   : ST depression induced by exercise relative to rest
  slp       : Slope of peak exercise ST segment
  caa       : Number of major vessels (0-4) colored by fluoroscopy
  thall     : Thalassemia (0-3)

TARGET VARIABLE (y):
==================================================
  output: Heart attack likelihood (1 = more likely, 0 = less likely)

Class Distribution:
output
1    165
0    124
Name: count, dtype: int64

Total samples: 289
Dataset Info:
age           int64
sex           int64
cp            int64
trtbps        int64
chol          int64
fbs           int64
restecg       int64
thalachh      int64
exng          int64
oldpeak     float64
slp           int64
caa           int64
thall         int64
output        int64
dtype: object

Missing Values:
age         0
sex         0
```

```
cp          0
trtbps      0
chol        0
fbs         0
restecg     0
thalachh    0
exng        0
oldpeak     0
slp         0
caa         0
thall       0
output      0
dtype: int64

Basic Statistics:
```

[1]:

```
           age     sex      cp  trtbps    chol     fbs  restecg  thalachh ␣
↪\
count  289.00  289.00  289.00  289.00  289.00  289.00   289.00    289.00
mean    54.01    0.68    1.02  131.38  247.96    0.15     0.52    150.23
std      9.13    0.47    1.03   17.52   51.60    0.35     0.51     22.90
min     29.00    0.00    0.00   94.00  126.00    0.00     0.00     71.00
25%     47.00    0.00    0.00  120.00  212.00    0.00     0.00    136.00
50%     54.00    1.00    1.00  130.00  243.00    0.00     1.00    154.00
75%     60.00    1.00    2.00  140.00  276.00    0.00     1.00    168.00
max     77.00    1.00    3.00  200.00  564.00    1.00     2.00    202.00

         exng  oldpeak     slp     caa   thall  output
count  289.00   289.00  289.00  289.00  289.00  289.00
mean     0.32     1.01    1.42    0.71    2.31    0.57
std      0.47     1.13    0.61    1.02    0.60    0.50
min      0.00     0.00    0.00    0.00    0.00    0.00
25%      0.00     0.00    1.00    0.00    2.00    0.00
50%      0.00     0.60    1.00    0.00    2.00    1.00
75%      1.00     1.60    2.00    1.00    3.00    1.00
max      1.00     6.20    2.00    4.00    3.00    1.00
```

Question 2

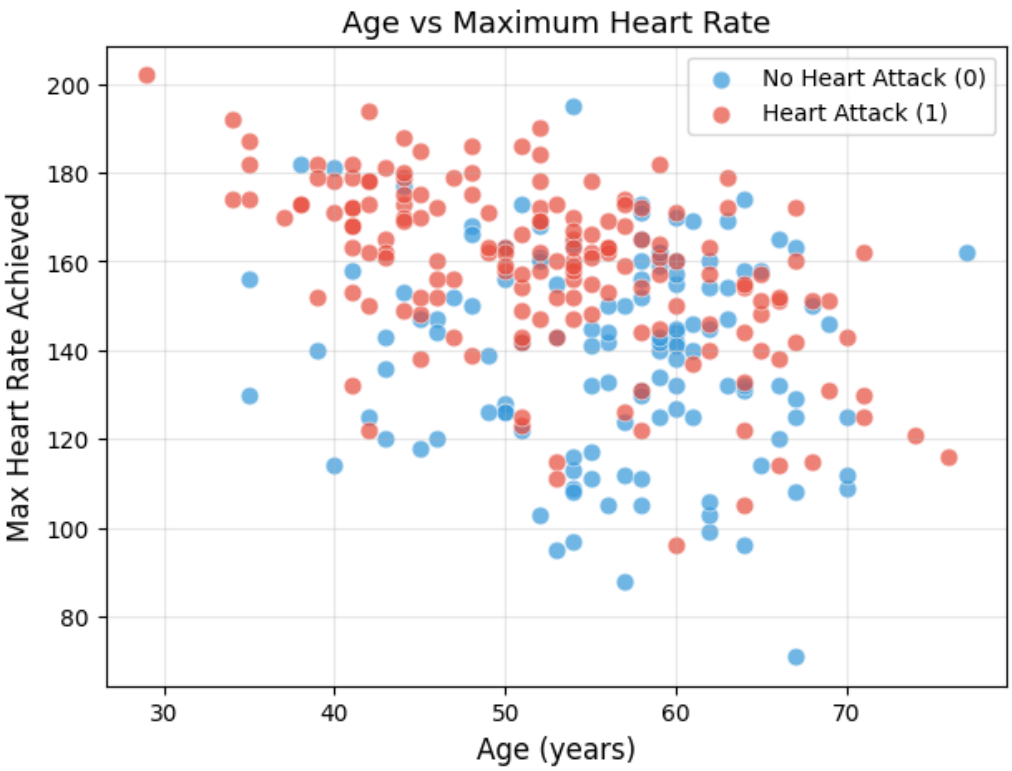
Create scatter plots for at least two pairs of features (e.g., Age vs Cholesterol, Age vs Maximum Heart Rate) to visualize class separability. Use different colors for each class.

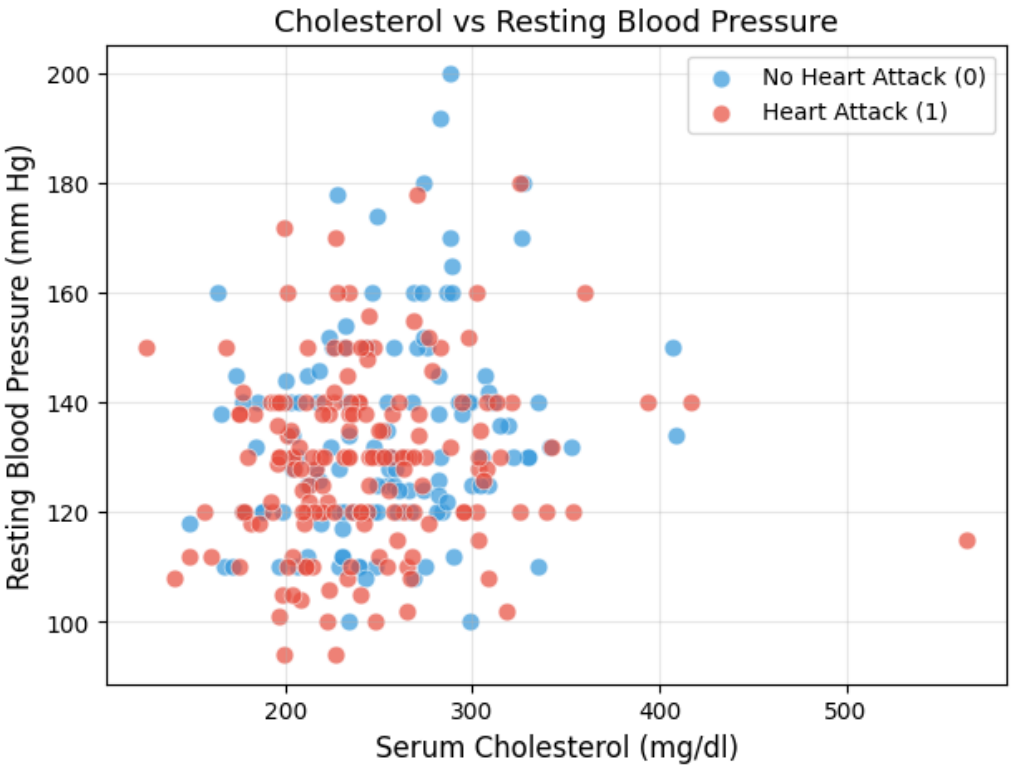
[2]:

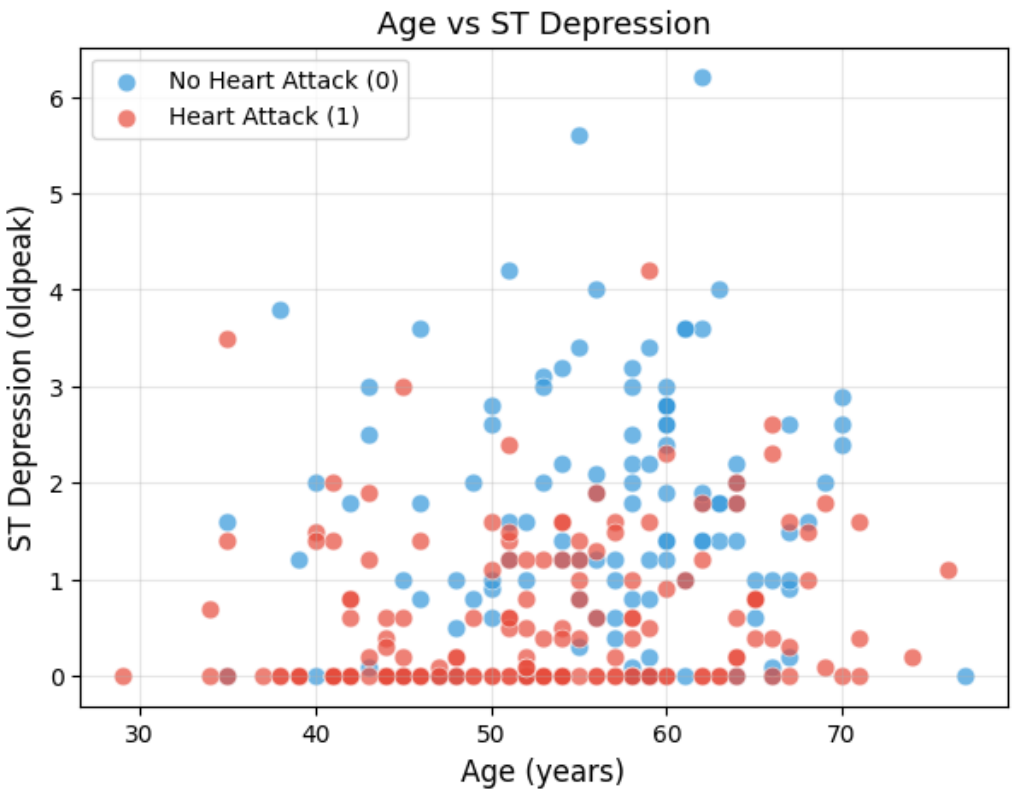
```python
# Color mapping for classes
colors = {1: '#e74c3c', 0: '#3498db'}
labels = {1: 'Heart Attack (1)', 0: 'No Heart Attack (0)'}

# -------- Plot 1: Age vs Max Heart Rate --------
plt.figure(figsize=(7,5))
for cls in [0,1]:
    subset = df[df['output'] == cls]
    plt.scatter(subset['age'], subset['thalachh'],
```

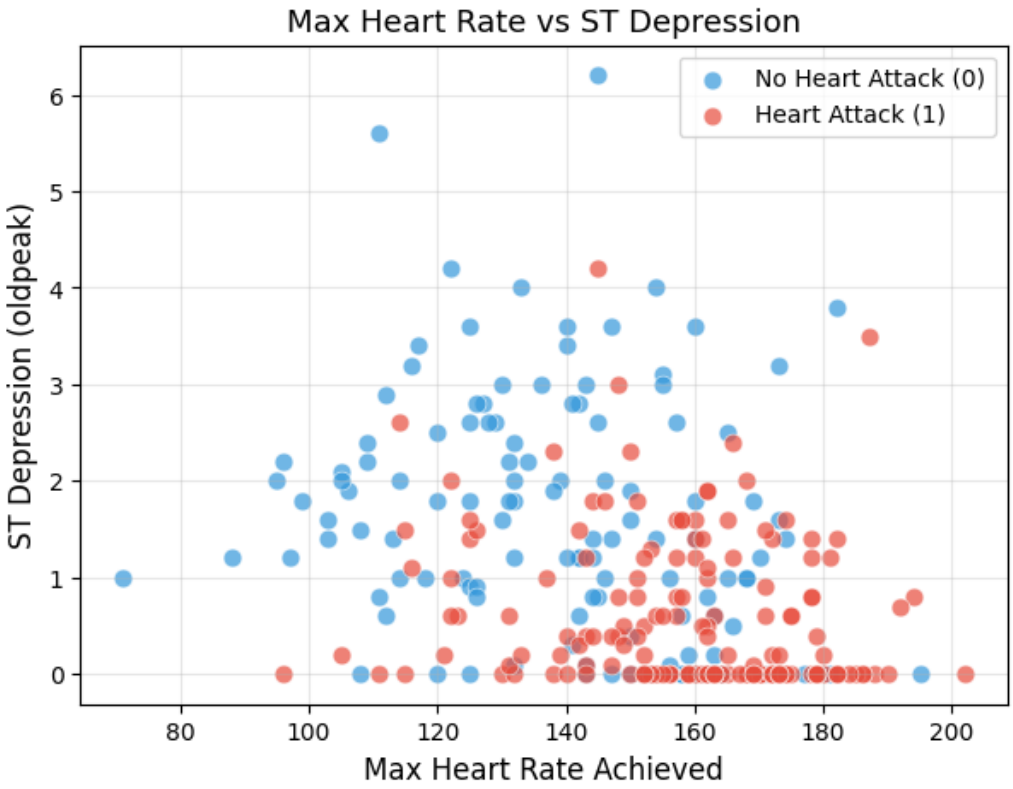
```
                c=colors[cls],
                label=labels[cls],
                alpha=0.7,
                edgecolors='white',
                linewidth=0.5,
                s=60)
plt.xlabel('Age (years)', fontsize=12)
plt.ylabel('Max Heart Rate Achieved', fontsize=12)
plt.title('Age vs Maximum Heart Rate', fontsize=13)
plt.legend()
plt.grid(True, alpha=0.3)

plt.savefig('Assignment_9b_scatter_age_vs_hr.png', dpi=1200,␣
 ↪bbox_inches='tight')
plt.show()
# -------- Plot 2: Cholesterol vs Resting Blood Pressure --------
plt.figure(figsize=(7,5))

for cls in [0,1]:
    subset = df[df['output'] == cls]
    plt.scatter(subset['chol'], subset['trtbps'],
                c=colors[cls],
                label=labels[cls],
                alpha=0.7,
                edgecolors='white',
                linewidth=0.5,
                s=60)

plt.xlabel('Serum Cholesterol (mg/dl)', fontsize=12)
plt.ylabel('Resting Blood Pressure (mm Hg)', fontsize=12)
plt.title('Cholesterol vs Resting Blood Pressure', fontsize=13)
plt.legend()
plt.grid(True, alpha=0.3)

plt.savefig('Assignment_9b_scatter_chol_vs_bp.png', dpi=1200,␣
 ↪bbox_inches='tight')
plt.show()



# -------- Plot 3: Age vs ST Depression --------
plt.figure(figsize=(7,5))

for cls in [0,1]:
    subset = df[df['output'] == cls]
    plt.scatter(subset['age'], subset['oldpeak'],
                c=colors[cls],
                label=labels[cls],
                alpha=0.7,
                edgecolors='white',
```

```
                linewidth=0.5,
                s=60)

plt.xlabel('Age (years)', fontsize=12)
plt.ylabel('ST Depression (oldpeak)', fontsize=12)
plt.title('Age vs ST Depression', fontsize=13)
plt.legend()
plt.grid(True, alpha=0.3)

plt.savefig('Assignment_9b_scatter_age_vs_oldpeak.png', dpi=1200,␣
 ↪bbox_inches='tight')
plt.show()



# -------- Plot 4: Max Heart Rate vs ST Depression --------
plt.figure(figsize=(7,5))

for cls in [0,1]:
    subset = df[df['output'] == cls]
    plt.scatter(subset['thalachh'], subset['oldpeak'],
                c=colors[cls],
                label=labels[cls],
                alpha=0.7,
                edgecolors='white',
                linewidth=0.5,
                s=60)

plt.xlabel('Max Heart Rate Achieved', fontsize=12)
plt.ylabel('ST Depression (oldpeak)', fontsize=12)
plt.title('Max Heart Rate vs ST Depression', fontsize=13)
plt.legend()
plt.grid(True, alpha=0.3)

plt.savefig('Assignment_9b_scatter_hr_vs_oldpeak.png', dpi=1200,␣
 ↪bbox_inches='tight')
plt.show()


print("All scatter plots saved at 1200 DPI")
```


Age vs Maximum Heart Rate
No Heart Attack (0)
Heart Attack (1)
Max Heart Rate Achieved
Age (years)
200
180
160
140
120
100
80
30
40
50
60
70


Cholesterol vs Resting Blood Pressure
No Heart Attack (0)
Heart Attack (1)
Resting Blood Pressure (mm Hg)
Serum Cholesterol (mg/dl)
200
180
160
140
120
100
200
300
400
500


Age vs ST Depression
No Heart Attack (0)
Heart Attack (1)
ST Depression (oldpeak)
Age (years)
0
1
2
3
4
5
6
30
40
50
60
70


Max Heart Rate vs ST Depression
No Heart Attack (0)
Heart Attack (1)
ST Depression (oldpeak)
Max Heart Rate Achieved
0
1
2
3
4
5
6
80
100
120
140
160
180
200

```
All scatter plots saved at 1200 DPI
```

Question 3

Build a Decision Tree classifier using sklearn.tree.DecisionTreeClassifier. Train the model and print the predicted classes for the test set.

[3]:
```python
# Prepare features and target
X = df.drop('output', axis=1)
y = df['output']

feature_names = X.columns.tolist()
print('Features used for training:', feature_names)
print('Target variable: output')

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f'\nTraining samples : {X_train.shape[0]}')
print(f'Testing samples  : {X_test.shape[0]}')
# Build and train the Decision Tree
dt_classifier = DecisionTreeClassifier(
    criterion='gini',
    max_depth=4,
    min_samples_split=5,
    min_samples_leaf=3,
    random_state=42
)

dt_classifier.fit(X_train, y_train)
print('Decision Tree trained successfully!')
print(f'Tree Depth: {dt_classifier.get_depth()}')
print(f'Number of Leaves: {dt_classifier.get_n_leaves()}')
print(f'Number of Features: {dt_classifier.n_features_in_}')
# Predict and display results
y_pred = dt_classifier.predict(X_test)

print('Predicted Classes for Test Set:')
print('='*60)
results_df = pd.DataFrame({
    'Actual':    y_test.values,
    'Predicted': y_pred,
    'Correct':   (y_test.values == y_pred)
})
print(results_df.to_string(index=False))

print(f'\nTotal Test Samples : {len(y_test)}')
print(f'Correct Predictions: {results_df["Correct"].sum()}')
print(f'Wrong Predictions  : {(~results_df["Correct"]).sum()}')
```

```
Features used for training: ['age', 'sex', 'cp', 'trtbps', 'chol', 'fbs',
'restecg', 'thalachh', 'exng', 'oldpeak', 'slp', 'caa', 'thall']
Target variable: output

Training samples : 231
Testing samples  : 58
Decision Tree trained successfully!
Tree Depth: 4
Number of Leaves: 14
Number of Features: 13
Predicted Classes for Test Set:
============================================================
 Actual  Predicted  Correct
      1          0    False
      0          0     True
      0          0     True
      0          1    False
      0          0     True
      0          1    False
      1          1     True
      0          0     True
      1          0    False
      0          0     True
      1          0    False
      1          0    False
      1          1     True
      1          1     True
      1          0    False
      1          1     True
      1          1     True
      1          0    False
      1          1     True
      1          1     True
      1          1     True
      0          0     True
      1          1     True
      0          0     True
      0          0     True
      0          1    False
      1          1     True
      0          0     True
      1          1     True
      0          1    False
      1          1     True
      1          1     True
      0          1    False
      0          1    False
      1          0    False
      0          0     True
      1          1     True
      1          1     True
```

```
      0           0     True
      1           1     True
      1           1     True
      1           1     True
      0           0     True
      0           1    False
      1           1     True
      0           0     True
      1           0    False
      0           0     True
      0           0     True
      1           1     True
      1           1     True
      0           0     True
      1           1     True
      1           1     True
      1           0    False
      0           0     True
      0           0     True
      1           1     True

Total Test Samples : 58
Correct Predictions: 42
Wrong Predictions  : 16
```

Question 4

Evaluate the Decision Tree classifier using accuracy, confusion matrix, and classification report.

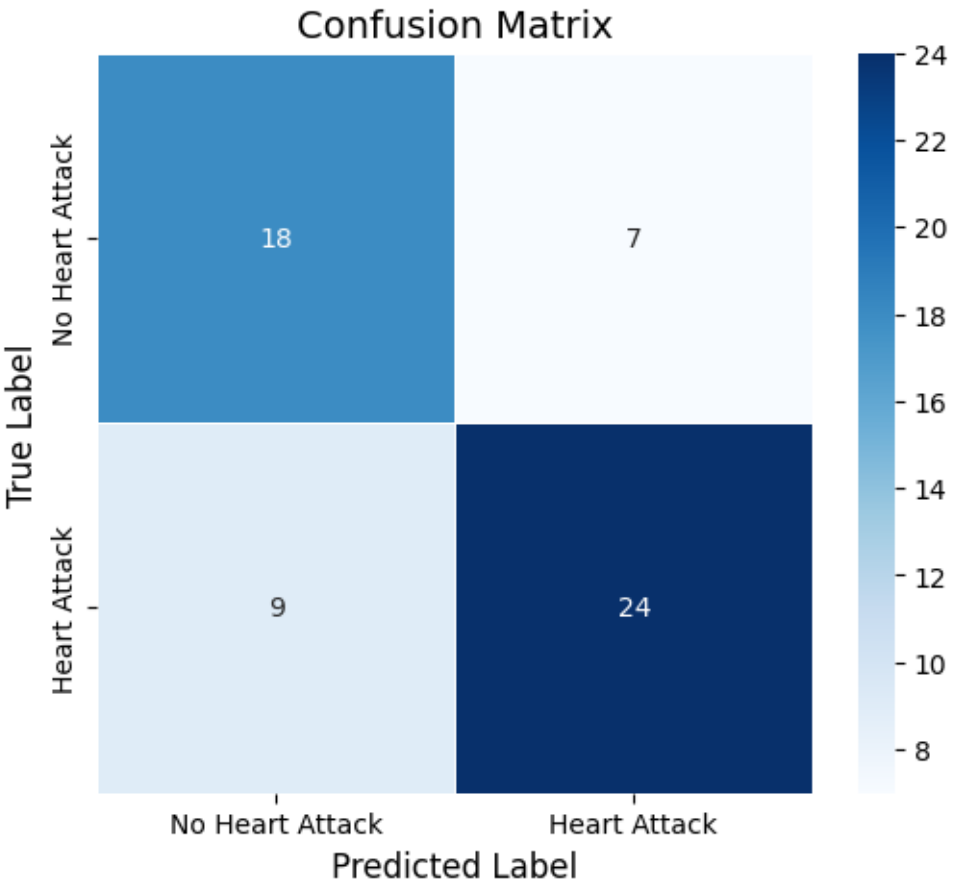
[4]:
```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy Score: {accuracy:.4f} ({accuracy*100:.2f}%)')
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print('\nConfusion Matrix:')
print(cm)
tn, fp, fn, tp = cm.ravel()
print(f'\n  True Negatives  (TN): {tn}')
print(f'  False Positives (FP): {fp}')
print(f'  False Negatives (FN): {fn}')
print(f'  True Positives  (TP): {tp}')

# Classification Report
print('\nClassification Report:')
print('='*55)
print(classification_report(y_test, y_pred,
                            target_names=['No Heart Attack (0)', 'Heart␣
 ↪Attack (1)']))
# -------- Confusion Matrix --------
plt.figure(figsize=(6,5))
```

```python
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['No Heart Attack', 'Heart Attack'],
    yticklabels=['No Heart Attack', 'Heart Attack'],
    linewidths=0.5,
    linecolor='white'
)

plt.title('Confusion Matrix', fontsize=14)
plt.xlabel('Predicted Label', fontsize=12)
plt.ylabel('True Label', fontsize=12)

plt.savefig('Assignment_9b_confusion_matrix.png', dpi=1200,␣
 ↪bbox_inches='tight')
plt.show()

print("Confusion matrix saved as Assignment_9b_confusion_matrix.png")

# -------- Classification Metrics Bar Chart --------
plt.figure(figsize=(7,5))

report = classification_report(
    y_test,
    y_pred,
    target_names=['No Heart Attack', 'Heart Attack'],
    output_dict=True
)

metrics = ['precision', 'recall', 'f1-score']
classes = ['No Heart Attack', 'Heart Attack']

x = np.arange(len(metrics))
width = 0.3
colors_bar = ['#3498db', '#e74c3c']

for i, (cls, color) in enumerate(zip(classes, colors_bar)):
    vals = [report[cls][m] for m in metrics]

    bars = plt.bar(
        x + i * width,
        vals,
        width,
        label=cls,
        color=color,
        alpha=0.85,
        edgecolor='white'
```

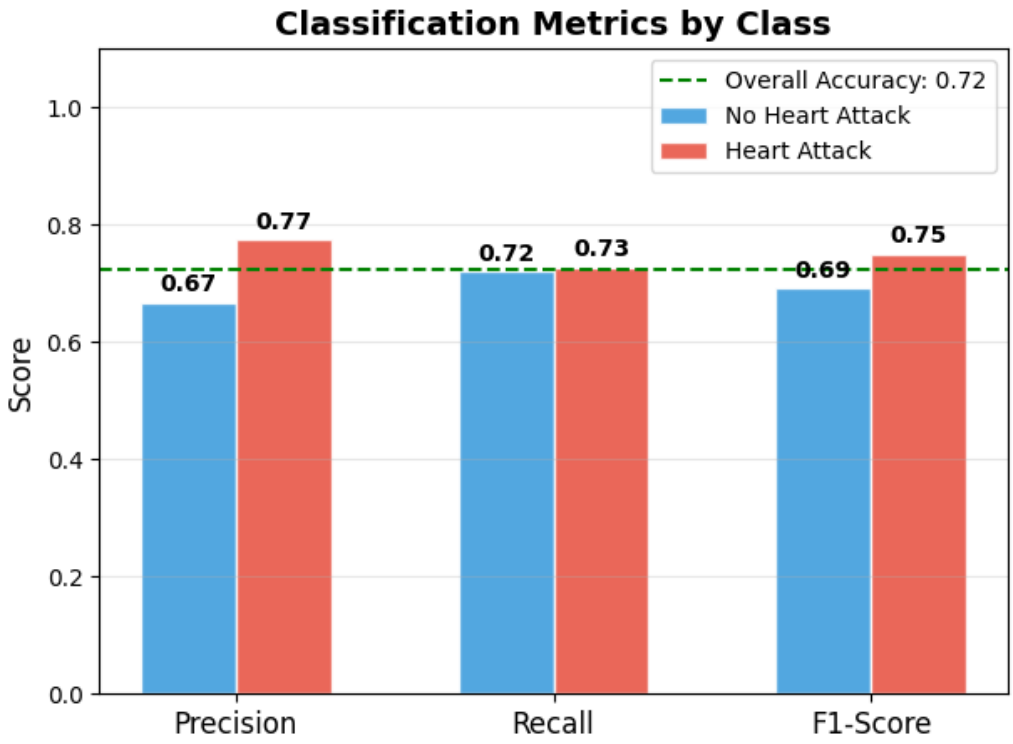
```python
    )

    for bar, val in zip(bars, vals):
        plt.text(
            bar.get_x() + bar.get_width()/2,
            bar.get_height() + 0.01,
            f'{val:.2f}',
            ha='center',
            va='bottom',
            fontsize=10,
            fontweight='bold'
        )
accuracy = accuracy_score(y_test, y_pred)
plt.axhline(
    y=accuracy,
    color='green',
    linestyle='--',
    linewidth=1.5,
    label=f'Overall Accuracy: {accuracy:.2f}'
)

plt.xticks(x + width/2, ['Precision', 'Recall', 'F1-Score'], fontsize=12)
plt.ylabel('Score', fontsize=12)
plt.ylim(0, 1.1)
plt.title('Classification Metrics by Class', fontsize=14,␣
 ↪fontweight='bold')

plt.legend(fontsize=10)
plt.grid(True, alpha=0.3, axis='y')

plt.savefig('Assignment_9b_classification_metrics.png', dpi=1200,␣
 ↪bbox_inches='tight')
plt.show()
```

```
Accuracy Score: 0.7241 (72.41%)

Confusion Matrix:
[[18  7]
 [ 9 24]]

  True Negatives  (TN): 18
  False Positives (FP): 7
  False Negatives (FN): 9
  True Positives  (TP): 24

Classification Report:
========================================================
                      precision    recall  f1-score   support

No Heart Attack (0)        0.67      0.72      0.69        25
```

```
Heart Attack (1)       0.77      0.73      0.75        33

        accuracy                           0.72        58
       macro avg       0.72      0.72      0.72        58
    weighted avg       0.73      0.72      0.73        58
```




```
Confusion matrix saved as Assignment_9b_confusion_matrix.png
```



Question 5

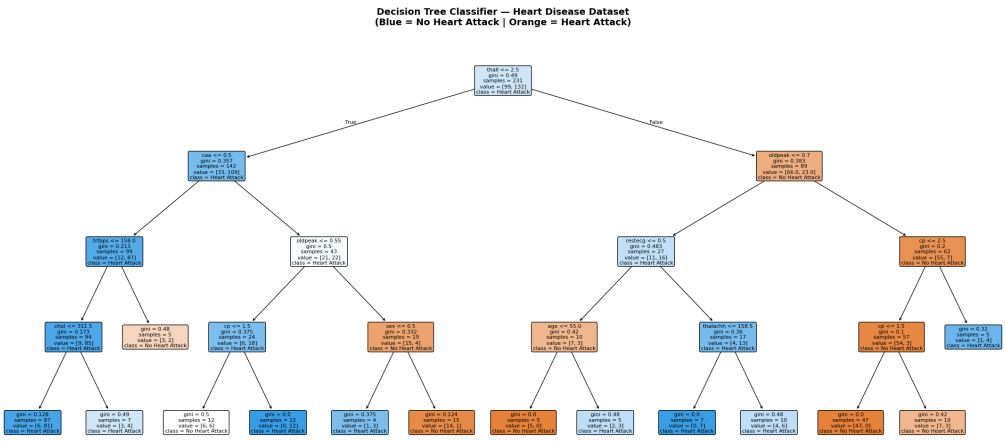
Visualize the trained decision tree using plot_tree and export it as text.

[5]:
```
print('Decision Tree (Text Representation):')
print('='*60)
tree_text = export_text(dt_classifier, feature_names=feature_names)
print(tree_text)
# --- Visual Decision Tree Plot ---
fig, ax = plt.subplots(figsize=(22, 10))

plot_tree(
    dt_classifier,
    feature_names=feature_names,
    class_names=['No Heart Attack', 'Heart Attack'],
    filled=True,
    rounded=True,
    fontsize=8,
    ax=ax,
    impurity=True,
    proportion=False
)

ax.set_title('Decision Tree Classifier - Heart Disease Dataset\n'
             '(Blue = No Heart Attack | Orange = Heart Attack)',
             fontsize=14, fontweight='bold', pad=20)
```

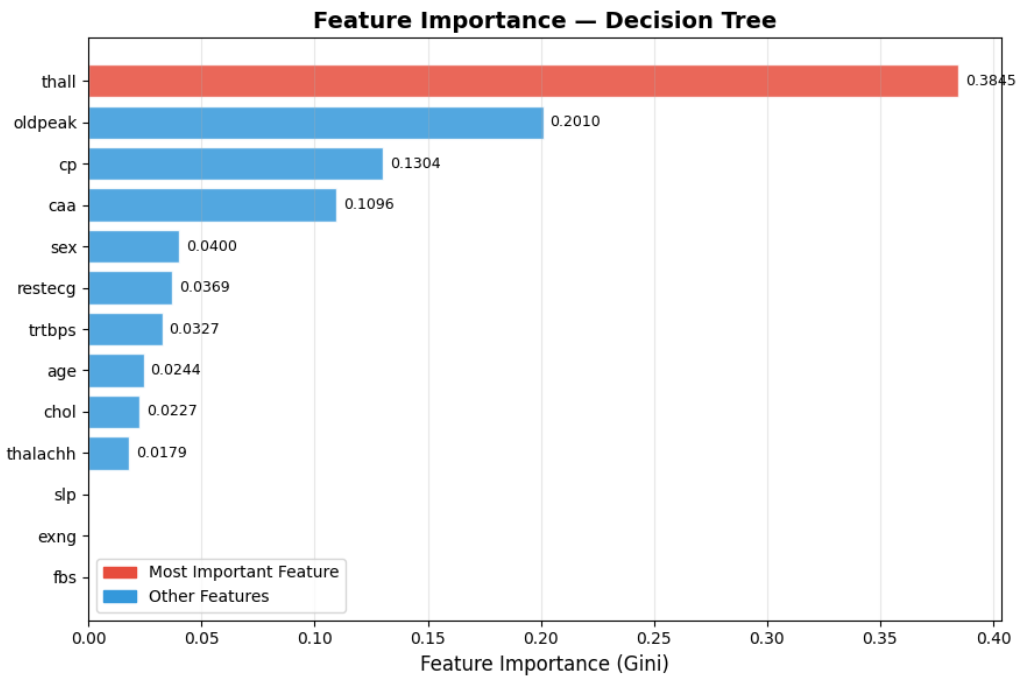
```python
plt.tight_layout()
plt.savefig('Assignment_9b_decision_tree_visual.png', dpi=1200,␣
↪bbox_inches='tight')
plt.show()
print('Decision tree visualization saved as␣
↪Assignment_9b_decision_tree_visual.png')
# Feature Importance
importances = dt_classifier.feature_importances_
feat_imp = pd.Series(importances, index=feature_names).
↪sort_values(ascending=True)

fig, ax = plt.subplots(figsize=(9, 6))
colors_imp = ['#e74c3c' if v == feat_imp.max() else '#3498db' for v in␣
↪feat_imp.values]
bars = ax.barh(feat_imp.index, feat_imp.values, color=colors_imp,␣
↪edgecolor='white', alpha=0.85)

for bar, val in zip(bars, feat_imp.values):
    if val > 0:
        ax.text(val + 0.003, bar.get_y() + bar.get_height()/2,
                f'{val:.4f}', va='center', fontsize=9)

ax.set_xlabel('Feature Importance (Gini)', fontsize=12)
ax.set_title('Feature Importance - Decision Tree', fontsize=14,␣
↪fontweight='bold')
ax.grid(True, alpha=0.3, axis='x')

red_patch = mpatches.Patch(color='#e74c3c', label='Most Important␣
↪Feature')
blue_patch = mpatches.Patch(color='#3498db', label='Other Features')
ax.legend(handles=[red_patch, blue_patch], fontsize=10)

plt.tight_layout()
plt.savefig('Assignment_9b_feature_importance.png', dpi=1200,␣
↪bbox_inches='tight')
plt.show()
print('Feature importance chart saved as Assignment_9b_feature_importance.
↪png')

print('\nTop 5 Most Important Features:')
print(feat_imp.sort_values(ascending=False).head())
```

```
Decision Tree (Text Representation):
============================================================
|--- thall <= 2.50
|   |--- caa <= 0.50
|   |   |--- trtbps <= 158.00
|   |   |   |--- chol <= 311.50
|   |   |   |   |--- class: 1
```

```
|   |   |   |--- chol >  311.50
|   |   |   |   |--- class: 1
|   |   |--- trtbps >  158.00
|   |   |   |--- class: 0
|   |--- caa >  0.50
|   |   |--- oldpeak <= 0.55
|   |   |   |--- cp <= 1.50
|   |   |   |   |--- class: 0
|   |   |   |--- cp >  1.50
|   |   |   |   |--- class: 1
|   |   |--- oldpeak >  0.55
|   |   |   |--- sex <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- sex >  0.50
|   |   |   |   |--- class: 0
|--- thall >  2.50
|   |--- oldpeak <= 0.70
|   |   |--- restecg <= 0.50
|   |   |   |--- age <= 55.00
|   |   |   |   |--- class: 0
|   |   |   |--- age >  55.00
|   |   |   |   |--- class: 1
|   |   |--- restecg >  0.50
|   |   |   |--- thalachh <= 158.50
|   |   |   |   |--- class: 1
|   |   |   |--- thalachh >  158.50
|   |   |   |   |--- class: 1
|   |--- oldpeak >  0.70
|   |   |--- cp <= 2.50
|   |   |   |--- cp <= 1.50
|   |   |   |   |--- class: 0
|   |   |   |--- cp >  1.50
|   |   |   |   |--- class: 0
|   |   |--- cp >  2.50
|   |   |   |--- class: 1
```



```
Decision tree visualization saved as Assignment_9b_decision_tree_visual.png
```




```
Feature importance chart saved as Assignment_9b_feature_importance.png

Top 5 Most Important Features:
thall      0.384527
oldpeak    0.200996
cp         0.130387
caa        0.109558
sex        0.039974
dtype: float64
```

Question 6

Predict the heart attack risk for a new patient with the following features: age=52, sex=1 (male), cp=0 (typical angina), trtbps=125, chol=212, fbs=0, restecg=1, thalachh=168, exng=0, oldpeak=1.0, slp=2, caa=2, thall=3

[6]:
```python
print('----- New Patient Prediction -----')

# New patient features
new_patient = pd.DataFrame([{
    'age': 52, 'sex': 1, 'cp': 0, 'trtbps': 125, 'chol': 212,
    'fbs': 0, 'restecg': 1, 'thalachh': 168, 'exng': 0,
    'oldpeak': 1.0, 'slp': 2, 'caa': 2, 'thall': 3
}])
```

```python
print('\nInput Features:')
print('  age=52, sex=1 (male), cp=0 (typical angina), trtbps=125,␣
↪chol=212,')
print('  fbs=0, restecg=1, thalachh=168, exng=0, oldpeak=1.0,')
print('  slp=2, caa=2, thall=3')

# Predict
pred_class = dt_classifier.predict(new_patient)[0]
pred_prob = dt_classifier.predict_proba(new_patient)[0]

outcome_map = {0: 'Less chance of heart attack', 1: 'More chance of heart␣
↪attack'}
print(f'\nPredicted Class: {pred_class} ({outcome_map[pred_class]})')
print('\nPrediction Probabilities:')
print(f'  Less chance (0):  {pred_prob[0]:.2f}')
print(f'  More chance (1):  {pred_prob[1]:.2f}')
print(f'\nResult: {outcome_map[pred_class].upper()}')
```

```
----- New Patient Prediction -----

Input Features:
  age=52, sex=1 (male), cp=0 (typical angina), trtbps=125, chol=212,
  fbs=0, restecg=1, thalachh=168, exng=0, oldpeak=1.0,
  slp=2, caa=2, thall=3

Predicted Class: 0 (Less chance of heart attack)

Prediction Probabilities:
  Less chance (0):  1.00
  More chance (1):  0.00

Result: LESS CHANCE OF HEART ATTACK
```